

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — ANNEXURE A

Technical Presentation — Solution Document

Course: Artificial Intelligence | Course Code: CSA17

Course Outcome: CO5 — Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications (BTL 5)

This document provides complete, presentation-ready solutions for all five questions in Annexure A: Reactive, Goal-Based, Utility-Based, Learning, and Multi-Agent Systems, each mapped to the official grading rubric.

Rubric Criterion	How this presentation addresses it
Technical Content Accuracy (30%)	Precise agent-theory grounding (PEAS, agent function, architecture) with correct AI terminology
Problem Identification (25%)	Clear problem statement, constraints, and engineering approach defined before the solution
Use of Tools (20%)	PySpark / RDD-based distributed processing pipeline with concrete transformations
Communication (15%)	Structured slide-ready sections, diagrams described, and a clear narrative flow
Professionalism (10%)	Ethical considerations and team-role notes included at the end of each topic

1 Reactive Agent-Based Traffic Control System

1.1 Problem Identification

Urban traffic intersections experience unpredictable, rapidly changing congestion caused by fluctuating vehicle inflow, pedestrian crossings, and emergency vehicle priority. Fixed-timer traffic signals cannot adapt to these real-time conditions, leading to avoidable congestion, fuel wastage, and delayed emergency response. The engineering problem is to design an agent that reacts instantly to live sensor data — without needing an internal model of the world or long-term planning — to minimise average vehicle wait time at each junction.

1.2 Why a Reactive Agent

A Reactive (simple reflex) agent is appropriate because traffic-light decisions are driven by condition–action rules evaluated on the current percept only: *if lane occupancy > threshold, extend green phase*. Speed of response matters more than deep reasoning, and the environment is fully observable through roadside sensors, making memory-less reflex behaviour sufficient and computationally cheap enough for real-time control.

1.3 PEAS Description

Performance Measure	Throughput, minimum average wait time, emergency-vehicle priority, safety
Environment	Road network, intersections, vehicles, pedestrians, weather
Actuators	Traffic signal lights, pedestrian signals, variable message signs
Sensors	Inductive loop detectors, IoT cameras, radar/LIDAR occupancy sensors

1.4 Agent Architecture

The agent follows the classic reflex loop: **Sensors** → **Condition–Action Rule Matcher** → **Actuators**. Each junction runs its own lightweight rule engine (e.g., a finite-state machine with states Green-N-S, Green-E-W, All-Red) so the system scales horizontally as more intersections are added, with no shared central bottleneck.

- **Percept:** per-lane vehicle count, average speed, pedestrian button state, emergency-vehicle beacon signal.
- **Rule base (examples):** occupancy > 80% → extend green by 10s; emergency beacon detected → force green on that approach; pedestrian request + no conflicting green → insert crossing phase.
- **Actuator command:** next signal phase and duration, sent to the local traffic controller.

1.5 Use of Tools — PySpark & RDD Pipeline

Although the reactive decision itself is local and low-latency, city-wide tuning of the rule thresholds requires distributed batch analysis of historical sensor logs using PySpark and RDDs:

```
raw_rdd = sc.textFile("hdfs://traffic/sensor_logs/*.csv")
parsed_rdd = raw_rdd.map(lambda line: line.split(",")) \
    .map(lambda f: (f[0], {"lane": f[1], "count": int(f[2]), "speed": float(f[3])}))
occupancy_by_junction = parsed_rdd.mapValues(lambda r: r["count"]) \
    .reduceByKey(lambda a, b: a + b)
peak_junctions = occupancy_by_junction.filter(lambda kv: kv[1] > THRESHOLD) \
    .sortBy(lambda kv: -kv[1])
peak_junctions.saveAsTextFile("hdfs://traffic/analysis/peak_junctions")
```

map parses raw CSV logs from thousands of intersections in parallel; **reduceByKey** aggregates lane occupancy per junction across the cluster; **filter** and **sortBy** identify congestion hot-spots. The resulting insights are fed back as updated threshold constants in the reflex rule base — a streaming pipeline (Spark Structured Streaming) can run the same logic on live data windows for near real-time recalibration.

1.6 Streaming Data Processing

- Sensor data is ingested via a message broker (e.g., Kafka) into micro-batches every 1–2 seconds.
- Spark Structured Streaming computes rolling occupancy averages per lane using windowed aggregation.
- Anomaly spikes (e.g., sudden queue build-up) trigger an immediate rule override sent to the local controller.
- Congestion prevention is achieved through the combination of instantaneous reflex control at the edge and periodic threshold tuning from the distributed batch layer.

1.7 Suggested Visuals for Slides

- Reflex-agent loop diagram: Sensors → Rules → Actuators.
- Intersection map with colour-coded congestion heat levels.
- PySpark pipeline flow: raw logs → map/reduceByKey → sorted congestion table.

1.8 Professionalism & Ethics Note

Deployment should include fail-safe defaults (revert to fixed timing on sensor failure), pedestrian-safety priority over throughput optimisation, and equitable service across low-traffic residential junctions, not just high-volume arterial roads.

2 Goal-Based Agent for Healthcare Diagnosis

2.1 Problem Identification

Clinicians must synthesise large volumes of patient data — symptoms, vitals, lab results, and history — to reach an accurate diagnosis and select a treatment pathway. A single reflex rule cannot capture this because the correct action depends on future consequences (patient outcome), not just the current percept. The problem is to design an agent that represents an explicit diagnostic *goal* (confirm a condition with sufficient confidence, or reach a stable outcome) and searches over possible tests/treatments to achieve it.

2.2 Why a Goal-Based Agent

A Goal-Based agent maintains an internal model of the patient's state and evaluates candidate actions by asking "will this get me closer to the goal?" rather than reacting blindly. This supports multi-step reasoning, e.g., ordering a test, updating belief state on the result, and choosing the next test — behaviour that a simple reflex agent cannot express.

2.3 PEAS Description

Performance Measure	Diagnostic accuracy, time-to-diagnosis, patient safety, cost of tests ordered
Environment	Electronic Health Records (EHR), lab systems, wearable/vital-sign monitors
Actuators	Recommend tests, suggest diagnosis, alert clinician, propose treatment pathway
Sensors	Patient symptoms input, lab result feeds, vital-sign streams, medical history

2.4 Agent Architecture & Decision Logic

The agent maintains a belief state (current hypotheses with confidence scores) and uses goal-directed search — conceptually similar to a decision tree / best-first search over possible next tests — to reduce diagnostic uncertainty until a goal state (diagnosis confidence $\geq$ threshold) is reached.

- **State representation:** vector of symptoms, vitals, prior test results, current candidate diagnoses with probabilities.
- **Goal test:** top hypothesis probability exceeds a clinically-validated confidence threshold, or all differential diagnoses except one are ruled out.
- **Action selection:** choose the next test that maximises expected information gain (reduces entropy over candidate diagnoses) — an explicit look-ahead rather than a fixed rule.
- **Plan output:** ordered sequence of recommended tests plus a suggested treatment pathway once the goal is reached.

2.5 Use of Tools — PySpark & RDD Pipeline

Historical de-identified patient records are processed at scale to build and validate the probability models the agent reasons over:

```
records_rdd = sc.textFile("hdfs://ehr/patient_records/*.json").map(json.loads)
clean_rdd = records_rdd.filter(lambda r: r.get("symptoms") and r.get("diagnosis"))
symptom_diag_rdd = clean_rdd.flatMap(
    lambda r: [(s, r["diagnosis"]), 1] for s in r["symptoms"])
cooccurrence = symptom_diag_rdd.reduceByKey(lambda a, b: a + b)
diagnosis_priors = cooccurrence.map(lambda kv: (kv[0][1], kv[1])) \
    .reduceByKey(lambda a, b: a + b).collectAsMap()
```

flatMap expands each patient record into (symptom, diagnosis) pairs; **reduceByKey** computes co-occurrence counts across millions of records in parallel, forming the statistical backbone (prior probabilities) the goal-based agent uses for information-gain calculations during reasoning. Spark MLlib can subsequently train a classifier on this RDD-derived feature set for more accurate goal-state confidence scoring.

2.6 Suggested Visuals for Slides

- Belief-state update diagram: symptom input → probability update → goal check.
- Decision tree snippet showing test-selection by information gain.
- PySpark pipeline diagram: EHR records → flatMap → reduceByKey → prior-probability table.

2.7 Professionalism & Ethics Note

The agent must operate as a clinical decision-support tool, not an autonomous diagnostician: final sign-off stays with a licensed clinician, patient data must be handled under HIPAA/DPDP-style privacy safeguards, and the model's confidence and reasoning trail should be explainable, not a black box.

3 Utility-Based Agent for E-Commerce Recommendations

3.1 Problem Identification

In large e-commerce catalogues, many products can satisfy a customer's stated goal (e.g., "buy a laptop"), but they differ in price, rating, delivery time, and relevance. Goal-based reasoning alone (goal achieved / not achieved) cannot distinguish between multiple valid options. The problem is to rank and recommend the option that maximises overall customer satisfaction, which requires quantifying trade-offs — i.e., a utility function.

3.2 Why a Utility-Based Agent

A Utility-Based agent assigns a numeric desirability score to each possible outcome and selects the action maximising expected utility, allowing it to balance competing factors such as price versus quality versus delivery speed — something a binary goal test cannot express.

3.3 PEAS Description

Performance Measure	Click-through rate, conversion rate, customer satisfaction, revenue per session
Environment	Product catalogue, user browsing/purchase history, inventory, pricing
Actuators	Display ranked product recommendations, personalised offers, bundle suggestions
Sensors	Clickstream events, search queries, cart activity, past purchase and rating data

3.4 Utility Function Design

For a candidate product p and user u , the agent computes:

$$U(p, u) = w_1 \cdot \text{Relevance}(p, u) + w_2 \cdot \text{Rating}(p) + w_3 \cdot (1 / \text{PriceNormalised}(p)) + w_4 \cdot \text{DeliverySpeedScore}(p) + w_5 \cdot \text{StockAvailability}(p)$$

Weights $w_1..w_5$ are tuned per business priorities (or learned from historical conversion data). The agent then ranks all candidate products by $U(p, u)$ and presents the top- k results, re-evaluating utility continuously as the user's session behaviour updates the relevance estimate.

3.5 Use of Tools — PySpark & RDD Pipeline

Utility scoring at catalogue scale (millions of products $\times$ millions of users) is computed with a distributed RDD pipeline:

```
clicks_rdd = sc.textFile("hdfs://ecom/clickstream/*.csv").map(lambda l: l.split(","))
user_product_rdd = clicks_rdd.map(
    lambda f: ((f[0], f[1]), {"clicks": 1, "rating": float(f[2]), "price": float(f[3])}))
agg_rdd = user_product_rdd.reduceByKey(
    lambda a, b: {"clicks": a["clicks"]+b["clicks"], "rating": max(a["rating"], b["rating"]),
    "price": a["price"]})
utility_rdd = agg_rdd.map(lambda kv: (kv[0], compute_utility(kv[1])))
top_recs = utility_rdd.groupBy(lambda kv: kv[0][0]) \
    .mapValues(lambda vals: sorted(vals, key=lambda x: -x[1])[:10])
```

map transforms raw clickstream rows into keyed (user, product) records; **reduceByKey** aggregates engagement signals in parallel across the cluster; the **compute_utility** function (broadcast to all workers) applies the weighted utility formula; **groupBy** + sorting produces the top-10 recommendations per user, which are pushed to a low-latency serving layer (e.g., Redis) for the storefront.

3.6 Suggested Visuals for Slides

- Utility function formula with weighted bar chart of factor contributions.
- Ranked product list showing utility scores side-by-side.
- PySpark pipeline: clickstream → reduceByKey → utility scoring → top-k recommendations.

3.7 Professionalism & Ethics Note

Utility weighting must avoid manipulative dark patterns (e.g., over-weighting margin at the expense of genuine relevance) and should be transparent about sponsored versus organic rankings to maintain customer trust.

4 Learning Agent for Personalized Education System

4.1 Problem Identification

Students differ widely in prior knowledge, learning pace, and preferred content difficulty. A static curriculum either bores advanced learners or overwhelms beginners. The problem is to design an agent that continuously observes student performance and adapts content difficulty and sequencing over time — behaviour that requires learning from feedback, not a fixed rule set or one-off utility calculation.

4.2 Why a Learning Agent

A Learning agent is composed of four classic sub-components — **Performance Element** (selects the next learning action), **Critic** (evaluates outcomes against a performance standard, e.g., quiz score), **Learning Element** (updates the internal model based on critic feedback), and **Problem Generator** (suggests exploratory content to gather more information about the student) — allowing performance to improve over time rather than remaining fixed.

4.3 PEAS Description

Performance Measure	Learning-outcome improvement, engagement/completion rate, mastery-level growth
Environment	Learning management system (LMS), content repository, assessment engine
Actuators	Serve next lesson/quiz, adjust difficulty, provide hints, recommend remedial content
Sensors	Quiz/assignment scores, time-on-task, click interactions, self-reported difficulty

4.4 Learning Agent Architecture

- **Performance Element:** given the student's current mastery-vector estimate, selects the next lesson/question difficulty using a policy (e.g., Item Response Theory or a bandit algorithm).
- **Critic:** compares quiz outcomes to expected performance and produces a reward/error signal.
- **Learning Element:** updates the mastery-vector model (e.g., Bayesian Knowledge Tracing or a lightweight neural model) based on the critic's signal, improving future predictions.
- **Problem Generator:** occasionally injects slightly harder or novel-topic questions to explore the student's true ability rather than only exploiting the current model (exploration vs. exploitation trade-off).

4.5 Use of Tools — PySpark & RDD Pipeline

Cross-student interaction logs are processed with Spark to retrain the shared knowledge-tracing model that seeds each student's personalised policy:

```
logs_rdd = sc.textFile("hdfs://lms/interactions/*.csv").map(lambda l: l.split(","))
student_topic_rdd = logs_rdd.map(
    lambda f: ((f[0], f[1]), (int(f[2]), 1))) # (student, topic) -> (correct, attempt)
mastery_rdd = student_topic_rdd.reduceByKey(
    lambda a, b: (a[0]+b[0], a[1]+b[1]))
mastery_scores = mastery_rdd.mapValues(lambda v: v[0] / v[1]) # accuracy per topic
difficulty_adjustments = mastery_scores.filter(lambda kv: kv[1] < 0.6) \
    .map(lambda kv: (kv[0], "recommend_remedial"))
```

map converts raw interaction logs into keyed (student, topic) attempt records in parallel; **reduceByKey** aggregates correct-answer counts across the distributed cluster to compute per-topic mastery; the resulting model feeds each student's Learning Element so that content adapts using patterns learned from the entire student population, not just one learner's history.

4.6 Suggested Visuals for Slides

- Four-block Learning Agent architecture diagram (Performance Element, Critic, Learning Element, Problem Generator).
- Line chart showing a student's mastery score improving over successive sessions.
- PySpark pipeline: interaction logs → reduceByKey → per-topic mastery scores → adaptive content trigger.

4.7 Professionalism & Ethics Note

Adaptive difficulty must avoid discouraging struggling students (no punitive difficulty spikes) and should keep learning analytics anonymised/aggregated where used for cross-student model training, respecting student data-privacy regulations.

5 Multi-Agent System for Disaster Management

5.1 Problem Identification

Disaster response (flood, earthquake, fire) requires simultaneous sensing across a wide geographic area, rapid situational assessment, and coordinated action among multiple responder teams — a scope too large and too time-critical for a single centralised agent to handle without becoming a bottleneck or single point of failure. The problem is to design a system of cooperating specialised agents that jointly detect events and coordinate response actions in real time.

5.2 Why a Multi-Agent System (MAS)

A MAS decomposes the problem into specialised, concurrently-operating agents that communicate and negotiate, giving scalability (agents can be added per region), robustness (no single point of failure), and faster local response (each agent acts near its own sensors) compared to a monolithic single-agent design.

5.3 Agent Roles (PEAS per role)

Agent Type	Role / Actuators	Sensors
Sensor Agent	Detects anomalies (water level, seismic activity, smoke, flood, seismic activity, and response satellite feeds, drone camera streams)	IoT flood/seismic/fire sensors, satellite feeds, drone camera streams
Decision Agent	Classifies severity, prioritises affected zones, allocates responder resources	Aggregates reports from Sensor Agents, resource-availability
Communication Agent	Routes alerts to responder teams, public warning messages, and lets agents	Message queues, inter-agent network connectivity status

5.4 Coordination Strategy

- **Communication protocol:** agents exchange structured messages (event type, location, severity, timestamp) over a publish–subscribe bus so any agent can react to relevant events without polling.
- **Task allocation:** a contract-net style negotiation lets Decision Agents 'bid' available responder resources for each incident, assigning the nearest/most capable team.
- **Conflict resolution:** when multiple Decision Agents claim overlapping resources, a priority rule (severity score, population at risk) arbitrates.
- **Data sharing:** a shared situational-awareness map is updated incrementally by all Sensor Agents so every Decision Agent reasons over a consistent, near-real-time picture.

5.5 Use of Tools — PySpark & RDD Pipeline

Because Sensor Agents across a region generate massive concurrent event streams, distributed processing is essential for fusing and prioritising incidents:

```
events_rdd = sc.textFile("hdfs://disaster/sensor_events/*.json").map(json.loads)
zone_events_rdd = events_rdd.map(
    lambda e: (e["zone_id"], (e["severity"], e["type"], e["timestamp"])))
severity_by_zone = zone_events_rdd.reduceByKey(
    lambda a, b: a if a[0] > b[0] else b) # keep highest-severity event per zone
priority_zones = severity_by_zone.filter(lambda kv: kv[1][0] >= CRITICAL_THRESHOLD) \
    .sortBy(lambda kv: -kv[1][0])
priority_zones.saveAsTextFile("hdfs://disaster/dispatch/priority_zones")
```

map parses concurrent JSON event streams from thousands of Sensor Agents in parallel; **reduceByKey** fuses multiple reports per zone into the single highest-severity signal across the cluster; **filter** and **sortBy** generate the ranked dispatch list that Decision Agents consume to allocate responder teams — turning raw distributed sensing into an actionable, prioritised response plan within seconds.

5.6 Suggested Visuals for Slides

- Multi-agent architecture diagram: Sensor Agents → message bus → Decision Agents → Communication Agents → responders.
- Regional map with colour-coded severity zones updated in real time.
- PySpark pipeline: event streams → reduceByKey (max severity) → prioritised dispatch list.

5.7 Professionalism & Ethics Note

The system must degrade gracefully under network loss (agents fall back to local autonomous action if disconnected from the coordination bus), avoid alarm fatigue from false positives, and ensure equitable resource allocation across all affected communities regardless of population density or economic status.

End of Document — All five topics are structured so each can be converted directly into 5–7 presentation slides: Title → Problem → PEAS/Architecture → Tools & Pipeline → Visuals → Ethics/Conclusion.